

Bài 9 : TẬP HỢP, BẢNG, TỪ ĐIỂN

Mục tiêu:

Về kiến thức:

- + Nắm vững định nghĩa tập hợp, các phép toán cơ bản của tập hợp.
- + Nắm vững các khái niệm từ điển, các phép toán thường gặp, các dạng biểu diễn từ điển.
- + Hiểu cấu trúc bảng băm: bảng băm đóng, bảng băm mở, cài đặt từ điển bởi bảng băm.

Về kỹ năng:

- + Sử dụng các kiến thức đã học để phân tích các bài toán ứng dụng.

9.1 Khái niệm tập hợp

Tập hợp trong toán học là một cấu trúc rời rạc cơ bản để từ đó dựng lên các cấu trúc rời rạc khác như: Các tổ hợp – là những tập hợp không sắp thứ tự của các phần tử; đồ thị – là tập hợp các đỉnh và các cạnh nối các đỉnh đó, cây – là một tập hợp các đỉnh và các cạnh nối hai đỉnh có quan hệ phân cấp, Tập hợp được dùng để mô hình hoá hay biểu diễn một nhóm bất kỳ các đối tượng trong thế giới thực cho nên nó đóng vai trò rất quan trọng trong mô hình hoá cũng như trong thiết kế các giải thuật.

- Tập hợp là một cấu trúc rời rạc được nói là *chứa* các phần tử của nó.
- Tập hợp thường dùng để gom nhóm các phần tử có các tính chất chung lại với nhau, nhưng nó cũng có thể chứa các phần tử chẳng có mối quan hệ gì với nhau.

Ví dụ 1 : $A = \{a, 2, \text{Fred}, \text{Lan}\}$.

Ví dụ 2: Tất cả các sinh viên vừa mới nhập trường lập nên một tập hợp, hoặc các sinh viên đang học môn CTDL & thuật toán lập nên một tập hợp, hoặc những sinh viên vừa mới nhập học và những sinh viên đang học môn CTDL & TT cũng lập nên một tập hợp

- Một tập hợp có thể là vô hạn hoặc hữu hạn

Để mô tả một tập hợp, trong toán học thường có các cách sau để xác định một tập hợp A :

- Dùng biểu đồ Ven là một đường cong khép kín, các điểm trong đường cong đó chỉ các phần tử của tập hợp.
- Liệt kê tất cả các phần tử của tập hợp A. Ví dụ: $A = \{1, 2, 3\}$
- Nêu lên các đặc trưng cho biết chính xác một đối tượng bất kỳ có là một phần tử của tập A hay không. Ví dụ: $A = \{x \mid x \text{ là số nguyên chẵn}\}$

9.2 Mô hình dữ liệu tập hợp

Trong thiết kế thuật toán, có thể sử dụng tập hợp như một mô hình dữ liệu. Khi đó, ngoài các phép toán hợp, giao, hiệu, chúng ta phải cần đến nhiều các phép toán khác. Sau đây chúng ta sẽ đưa ra một số phép toán cơ bản quan trọng nhất, các phép toán này sẽ được mô tả bởi các *thủ tục* hoặc *hàm*:

- 1- Thủ tục **MAKENULL(A)**: khởi tạo tập A rỗng
- 2- Thủ tục **UNION(A,B,C)** nhận vào 3 tham số là A,B,C; Thực hiện phép toán lấy hợp của hai tập A và B và trả ra kết quả là tập hợp $C = A \cup B$.
- 3- Thủ tục **INTERSECTION(A,B,C)** nhận vào 3 tham số là A,B,C; Thực hiện phép toán lấy giao của hai tập A và B và trả ra kết quả là tập hợp $C = A \cap B$.
- 4- Thủ tục **DIFFERENCE(A,B,C)** nhận vào 3 tham số là A,B,C; Thực hiện phép toán lấy hợp của hai tập A và B và trả ra kết quả là tập hợp $C = A \setminus B$
- 5- Hàm **MEMBER(x,A)** cho kết quả kiểu logic (đúng/sai) tùy theo x có thuộc A hay không. Nếu $x \in A$ thì hàm cho kết quả là đúng, ngược lại cho kết quả sai.
- 6- Thủ tục **INSERTSET(x,A)** thêm x vào tập hợp A
- 7- Thủ tục **DELETESSET(x,A)** xóa x khỏi tập hợp A
- 8- Thủ tục **ASSIGN(A,B)** gán A cho B (tức là $B:=A$)

9- Hàm **MIN(A)** cho phần tử bé nhất trong tập A

10- Hàm **EQUAL(A,B)** cho kết quả TRUE nếu $A=B$ ngược lại cho kết quả FALSE

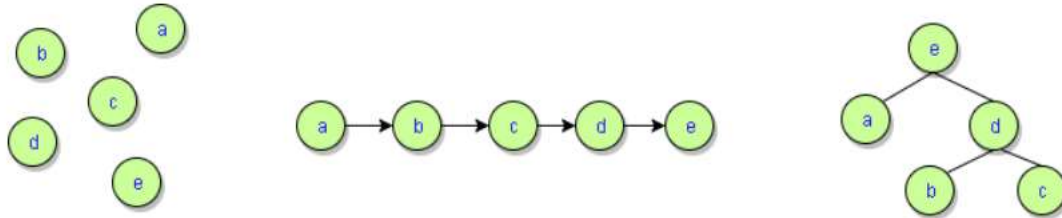
11- Thủ tục **INPUT(A)**: nhập dữ liệu cho tập A

12- Thủ tục **OUTPUT(A)**: xuất dữ liệu tập hợp A lên màn hình

9.3 Các cấu trúc dữ liệu tập hợp

Tập hợp có thể được cài đặt sử dụng các cấu trúc dữ liệu khác nhau. Một số cấu trúc được thiết kế để cải tiến hiệu quả thời gian thực thi của các thao tác trong khi các cấu trúc khác lại đặt trọng tâm hơn vào không gian lưu trữ, việc lựa chọn cấu trúc dữ liệu tập hợp phù hợp với bài toán phụ thuộc nhiều vào các thao tác cụ thể và các đặc tính của tập hợp (ví dụ kích cỡ) mà bài toán quan tâm.

Tập hợp có thể được biểu diễn thông qua các cấu trúc dữ liệu hàng đợi, danh sách, cây, bản đồ, bảng băm, ... Hình 9.1 mô tả hình ảnh một tập hợp gồm 5 ký tự được biểu diễn bởi hai cấu trúc dữ liệu là danh sách và cây nhị phân.



Hình 9.1: Một số cấu trúc dữ liệu tập hợp

Một số cấu trúc dữ liệu tập hợp cơ bản:

- Biểu diễn tập hợp bởi vector bit;
- Biểu diễn tập hợp bởi danh sách liên kết;
- Biểu diễn tập hợp bởi cây;
- Biểu diễn tập hợp bởi bảng băm.

Với mỗi dạng biểu diễn tập hợp hay cấu trúc dữ liệu tập hợp, các thao tác trên tập hợp được cài đặt tương ứng với cấu trúc dữ liệu đó. Ví dụ, với tập hợp được biểu diễn bởi

danh sách liên kết đơn, phép toán UNION(A,B,C) tương như phép toán nối hai danh sách liên kết đơn A, B thành danh sách liên kết đơn C với sự loại bỏ các phần tử trùng lặp. Bạn đọc có thể tham khảo cấu trúc dữ liệu danh sách liên kết đơn để cài đặt phép toán này. Phép toán MEMBER(x,A) sẽ tương tự như phép toán tìm kiếm phần tử x trong danh sách liên kết A,

9.3.1 Cài đặt tập hợp bởi vector bit

Giả sử, xét tập hợp A gồm các số nguyên thuộc phạm vi từ 1 đến n (hoặc được mã hóa thành các số nguyên thuộc phạm vi từ 1 -> n). Khi đó ta có thể dùng vector bit (mảng boolean) để biểu diễn tập A : (A[1], A[2], ... ,A[n]), trong đó thành phần thứ i : A[i] = true nếu $i \in A$, A[i] = false nếu $i \notin A$

Ví dụ: Giả sử các phần tử của tập hợp được lấy trong các số nguyên từ 1 đến 10 , khi đó tập hợp được biểu diễn bởi một mảng một chiều có 10 phần tử với các giá trị phần tử thuộc kiểu logic. Chẳng hạn tập hợp $A=\{1,3,5,8\}$ được biểu diễn trong mảng có 10 phần tử như sau:

Bảng 9.1: Biểu diễn tập hợp A bởi véc tơ bit gồm 10 phần tử

ID	0	1	2	3	4	5	6	7	8	9
Content	1	0	1	0	1	0	0	1	0	0

*Khai báo tập hợp:

```
#include<stdio.h>
#include<conio.h>

#define N 100 // kích co tap hop
typedef int item;// kieu phan tu cua tap hop
typedef item Set[N];
```

*Một số thao tác cơ bản:

0 - Thủ tục MAKENULL(A): khởi tạo tập A rỗng.

```

void MakeNull(Set A) {
    for(int i=0; i<N; i++)
        A[i]=0;
}

```

1- Thủ tục UNION(A,B,C) nhận vào 3 tham số là A,B,C; Thực hiện phép toán lấy hợp của hai tập A và B và trả ra kết quả là tập hợp $C = A \cup B$.

```

void Union(Set A, Set B, Set C) {
    for (int i=0; i<N; i++)
        if ((A[i]==1) || (B[i]==1))
            C[i]=1;
        else
            C[i]=0;
}

```

2 -Thủ tục INTERSECTION(A,B,C) nhận vào 3 tham số là A,B,C; Thực hiện phép toán lấy giao của hai tập A và B và trả ra kết quả là tập hợp $C = A \cap B$.

```

void Intersection(Set A, Set B, Set C) {
    for (int i=0; i<N; i++)
        if ((A[i]==1) && (B[i]==1))
            C[i]=1;
        else
            C[i]=0;
}

```

3 - Thủ tục DIFFERENCE(A,B,C) nhận vào 3 tham số là A,B,C; Thực hiện phép toán lấy hợp của hai tập A và B và trả ra kết quả là tập hợp $C = A \setminus B$

```

void Difference(Set A, Set B, Set C) {
    for (int i=0; i<N; i++)

        if ((A[i]==1) && (B[i]==1))
            C[i]=0;
        else
            C[i]=A[i];
}

```

4 - Hàm MEMBER(x,A) cho kết quả kiểu logic (đúng/sai) tùy theo x có thuộc A hay không. Nếu $x \in A$ thì hàm cho kết quả là đúng, ngược lại cho kết quả sai.

```

item Member(item x, Set A) {
    return A[x]==1;
}

```

5 - Thủ tục INSERTSET(x,A) thêm x vào tập hợp A

```
void Insert(item x, Set A) {  
    A[x] = 1;  
}
```

6 - Thủ tục DELETESet(x,A) xóa x khỏi tập hợp A

```
void Delete(item x, Set A) {  
    A[x] = 0;  
}
```

7 - Thủ tục ASSIGN(A,B) gán các phần tử A cho B

```
void Assign( Set A, Set B) {  
    for (int i=0; i<N; i++)  
        B[i] = A[i];  
}
```

8 - Hàm MIN(A) cho phần tử bé nhất trong tập A

```
item Min(Set A){  
    int i = 0, Found = -1;  
    while(i<N && Found == -1)  
        if(A[i]==1)  
            Found = i;  
        else  
            i++;  
    return Found;  
}
```

9 - Hàm EQUAL(A,B) cho kết quả TRUE nếu A=B ngược lại cho kết quả FALSE

```
item Equal(Set A, Set B) {  
    int i = 0;  
    Found = 0;  
    while(i<N && Found == 0)  
        if(A[i] != B[i])  
            Found = 1;  
        else  
            i++;  
    return Found;  
}
```

10 - Thủ tục INPUT(A): nhập dữ liệu cho tập A

```

void Input(Set A) {
    int x = 0;
    printf("\nNhap den -1 thi ket thuc: ");
    do {
        scanf("%d",&x);
        if(x!=-1)
            A[x] = 1;
    }while(x!=-1);
}

```

11 - Thủ tục OUTPUT(A): xuất dữ liệu tập hợp A lên màn hình

```

void Output(Set A) {
    for (int i=0; i<MaxLength; i++)
        if(A[i]==1)
            printf("%3d",i);
}

```

Nhận xét:

Cấu trúc dữ liệu tập hợp vector bit thuận lợi cho việc cài đặt các thao tác, độ phức tạp tính toán của các thao tác là $O(n)$, tuy nhiên nó thường không phù hợp với các tập hợp có kích cỡ quá lớn. Ví dụ thêm x vào A chỉ việc cho $A[x-1] = \text{true}$, để xác định xem x có là phần tử của tập A không ta chỉ cần biết $A[x]$ là 1/true hay 0/false.

9.3.2 Cài đặt tập hợp bởi danh sách liên kết

Tập hợp cũng có thể cài đặt bằng danh sách liên kết, trong đó mỗi phần tử của danh sách là một thành viên của tập hợp. Không như biểu diễn bằng vector bit, sự biểu diễn này dùng kích thước bộ nhớ tỉ lệ với số phần tử của tập hợp chứ không phải là kích thước đủ lớn cho toàn thể các tập hợp đang xét. Hơn nữa, ta có thể biểu diễn một tập hợp bất kỳ. Mặc dù thứ tự của các phần tử trong tập hợp là không quan trọng nhưng nếu một danh sách liên kết có thứ tự nó có thể trợ giúp tốt cho các phép duyệt danh sách. Chẳng hạn nếu tập hợp A được biểu diễn bằng một danh sách có thứ tự tăng thì hàm $\text{MEMBER}(x,A)$ có thể thực hiện việc so sánh x một cách tuần tự từ đầu danh sách cho đến khi gặp một phần tử $y \geq x$ chứ không cần so sánh với tất cả các phần tử trong tập hợp.

Một ví dụ khác, chẳng hạn ta muốn tìm giao của hai tập hợp A và B có n phần tử. Nếu A, B biểu diễn bằng các danh sách liên kết chưa có thứ tự thì để tìm giao của A và B ta phải tiến hành như sau: for (mỗi x thuộc A), duyệt danh sách B xem x có thuộc B

không. Nếu có thì x thuộc giao của hai tập hợp A và B . Rõ ràng quá trình này có thể phải cần đến $n \times m$ phép kiểm tra (với n, m là độ dài của A và B).

Nếu A, B được biểu diễn bằng danh sách có thứ tự tăng thì đối với một phần tử $e \in A$ ta chỉ tìm kiếm trong B cho đến khi gặp phần tử $x \geq e$. Quan trọng hơn nếu f đứng ngay sau e trong A thì để tìm kiếm f trong B ta chỉ cần tìm từ phần tử x trở đi chứ không phải từ đầu danh sách lưu trữ tập hợp B .

Việc cài đặt tập hợp bởi danh sách liên kết sẽ khắc phục hạn chế về không gian khi sử dụng mảng. Tuy nhiên trong cách cài đặt này, việc thực hiện các phép toán trên tập hợp sẽ phức tạp hơn.

***Khai báo cấu trúc**

```
typedef int item;
typedef struct Node{
    item info;
    Node *Next;
}
typedef Node *pointer;
typedef pointer Set;
```

9.4 Từ điển (Dictionary)

9.4.1 Khái niệm

Từ điển là một kiểu dữ liệu trừu tượng, là một tập hợp đặc biệt, trong đó chúng ta chỉ quan tâm đến các phép toán InsertSet, DeleteSet, Member và MakeNullSet. Sở dĩ chúng ta nghiên cứu từ điển là do trong nhiều ứng dụng không sử dụng đến các phép toán hợp, giao, hiệu ... của hai tập hợp. Ngược lại ta cần một cấu trúc làm sao cho việc tìm kiếm, thêm và bớt phần tử có phần hiệu quả nhất gọi là từ điển. Chúng ta cũng chấp nhận MakeNullSet như là phép khởi tạo cấu trúc từ điển.

9.4.2 Các phương pháp cài đặt từ điển

Từ điển là một tập hợp, do đó chúng ta phải sử dụng các phương pháp cài đặt tập hợp để cài đặt từ điển:

- Bảng vector bit: Sử dụng phương pháp này khi từ điển là tập hợp gồm các phần tử có thể dùng làm tập chỉ số cho mảng (thuộc kiểu dữ liệu vô hướng đếm được hoặc được

mã hóa thành kiểu dữ liệu vô hướng đếm được)

- Bảng danh sách (kế tiếp hoặc móc nối), không thuận lợi cho phép tìm kiếm (với danh sách móc nối vì phải truy cập tuần tự -> chỉ thích hợp khi áp dụng phương pháp tìm kiếm tuần tự), không thuận lợi cho phép toán thêm vào và lấy ra (với danh sách kế tiếp – vì có hiện tượng co, dãn, dịch chuyển các phần tử khác)

Giả sử từ điển là danh sách được sắp thứ tự tuyến tính -> sử dụng cây tìm kiếm nhị phân: Độ phức tạp về mặt thời gian của các phép toán trên từ điển là lớn (tương tự như danh sách) trong trường hợp cây suy biến thành danh sách.

Ta cũng có thể sử dụng cây cân bằng để biểu diễn tập hợp: Ưu điểm là không xảy ra trường hợp suy biến như cây TKNP, tuy nhiên các phép toán loại bỏ và xen vào trên cây cân bằng khá phức tạp, vì phải cân bằng lại cây – Về cây cân bằng, bạn đọc có thể tham khảo trong các tài liệu tham khảo của giáo trình

Do đó, để biểu diễn từ điển thuận lợi cho các phép toán, và có thể áp dụng cho tập hợp có kích cỡ lớn (từ điển thường có kích thước lớn), ta xét cách cài đặt khác đó là: sử dụng CTDL bảng băm để cài đặt từ điển.

9.4.3 Cấu trúc dữ liệu bảng băm

Bảng băm là một cấu trúc dữ liệu rất thích hợp cho việc biểu diễn một tập hợp có số phần tử lớn (ví dụ: từ điển), ở đó dữ liệu được lưu trữ theo định dạng mảng và mỗi giá trị có giá trị khóa duy nhất. Việc truy cập sẽ trở lên rất nhanh nếu ta biết chỉ số của phần tử cần tìm.

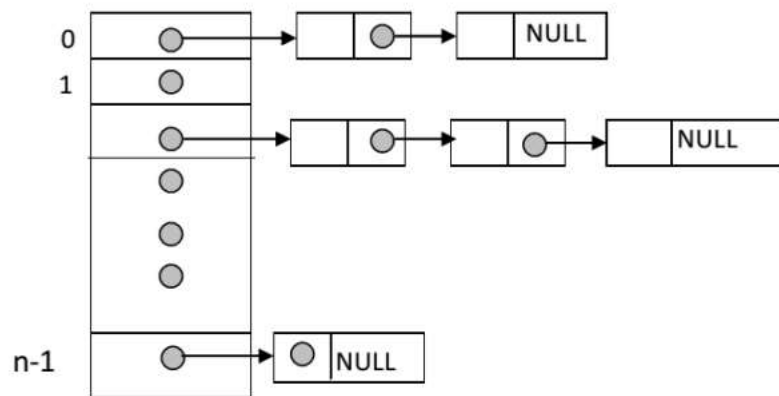
Bảng băm sử dụng mảng để lưu trữ các phần tử và sử dụng kỹ thuật băm để sinh khóa cho mỗi phần tử (khóa) được chèn vào bảng. Có 2 phương pháp băm khác nhau:

- Phương pháp Băm mở: Cho phép sử dụng một không gian không hạn chế để lưu giữ các phần tử của tập hợp.
- Phương pháp băm đóng: Sử dụng một không gian cố định và do đó tập hợp được cài đặt phải có cỡ không vượt quá không gian cho phép

9.4.4 Cài đặt từ điển bằng bảng băm

* Định nghĩa bảng băm mở

Tư tưởng cơ bản của bảng Băm là: Phân chia một tập hợp đã cho thành một số cố định các lớp (N lớp) được đánh số 0, 1, ..., N-1. Sử dụng mảng T với chỉ số chạy từ 0 đến N-1. Mỗi thành phần T[i] chứa một con trỏ, trỏ tới phần tử đầu tiên của danh sách chứa các phần tử của tập hợp thuộc lớp thứ i. Các phần tử của tập hợp thuộc mỗi lớp được tổ chức dưới dạng một danh sách liên kết, mỗi danh sách được gọi là con tr “rổ”, T được gọi là bảng băm (hash table, Hình 7.2).



Hình 9.2: Cấu trúc bảng băm mở

Việc chia các phần tử của tập hợp vào các lớp được thực hiện bởi hàm băm (hashCost)

* Hàm băm:

Băm là một kỹ thuật chuyển đổi một miền các giá trị khóa vào một miền gồm các chỉ số của mảng. Kỹ thuật này có thể xem như một hàm (hàm băm) ánh xạ từ tập dữ khóa A đến các số nguyên (tập chỉ số) 0..N-1:

$$\text{hashCost}: A \rightarrow 0..N-1;$$

Theo đó giả sử $x \in A$ thì $h(x)$ là một số nguyên sao cho: $0 \leq \text{hashCost}(x) \leq N-1$.

Hai tiêu chuẩn chính để lựa chọn một hàm băm là (1) hàm băm phải cho phép tính được dễ dàng và nhanh chóng giá trị Băm của mỗi khóa; (2) phải phân bố đều các khóa

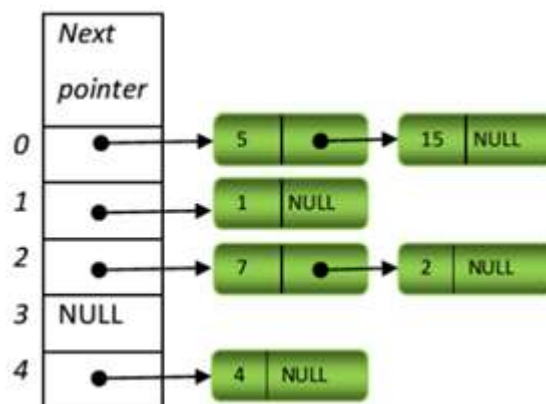
vào các rổ. Trong các phương pháp xây dựng hàm băm như băm gấp, băm bỏ bớt, .. thì phương pháp băm bằng cách chia lấy phần dư được xem là phổ biến.

Hàm băm lấy phần dư được thiết kế như sau:

```
int hashCode(int key){
    return
        key % SIZE;
}
```

Ví dụ : xét tập hợp $A = \{1, 5, 7, 2, 5, 15\}$, giả sử bảng băm A gồm 5 lớp, khi đó hàm băm là $\text{hashCost}(x) = x\%5$. Hình 7.3 mô tả cấu trúc của A được biểu diễn bởi bảng băm mở. Mỗi phần tử của bảng là một con trỏ (pointer) trỏ tới phần tử đầu tiên của rổ chứa các phần tử là chỉ số mảng của phần tử đó. Mỗi rổ được biểu diễn như một danh sách liên kết đơn, trong đó mỗi nút trong danh sách là một bản ghi gồm 2 trường: key (tương ứng với khóa của phần tử); next là con trỏ trỏ tới phần tử tiếp theo trong danh sách.

- Cài đặt từ điển bởi bảng băm mở
- Mỗi phần tử trong rổ có thể được mô tả theo cấu trúc sau:



Hình 9.3: Cấu trúc dữ liệu bảng băm mở biểu diễn từ điển/tập hợp A

```
typedef struct member{
    char *key ;
    // cac truong thong tin khac (neu co)
    struct member *next ;
} NodeT;
```

Sau đó, bảng băm T là một mảng gồm N con trỏ, mỗi con trỏ trỏ tới phần tử đầu tiên

của rổ tương ứng.

```
NodeT *T[N] ;
```

– Cài đặt các thao tác cơ bản của từ điển:

1- Hàm SEARCH (key, T)

Để kiểm tra xem phần tử có khóa x nào đó có trong từ điển T hay không, trước hết ta cần xác định giá trị băm của x, $h(x)$. Theo cấu trúc của bảng băm thì khóa x sẽ nằm trong rổ được trỏ bởi $T[h(x)]$. Việc tìm kiếm phần tử có khóa x trong rổ này được thực hiện tương tự như tìm kiếm phần tử x trong danh sách liên kết đơn.

Giải thuật tìm kiếm trả về 1 nếu tìm thấy, ngược lại trả về 0:

```
struct int search(char *x, NodeT *T[N]){  
  
    struct member *p;  
    int h = hashCode(x); // gia tri bam  
    p=T[h]; // bat dau tim kiem  
    while(p != NULL){  
        if(p->key == x)  
            return 1;  
        p=p->next;  
    }  
    return 0;  
}
```

2- Thủ tục INSERT(key [,data], T)

Để thêm một phần tử có khóa x vào từ điển T ta phải tính $h(x)$ để xác định rổ sẽ chứa x, cấp phát ô nhớ chứa phần tử cần thêm, đưa phần tử cần thêm vào rổ. Vì ta không quan tâm đến thứ tự các phần tử trong mỗi rổ nên ta để đơn giản ta thêm phần tử mới ngay đầu rổ này.

```
struct void insert(char *x [,data], NoteT *T[N]){  
    // cap phat o nho chua phan tu co khoa x  
    struct member *p = (struct member*) malloc(sizeof(struct member));  
    p->data = data; // neu co  
    p->key = x;  
    int h = hashCode(x);  
    // them phan tu co khoa x vao ro (them vao dau ds tuong ung voi ro)  
    p->next=T[h];  
    T[h]=p;  
}
```

3- Thủ tục DELETE(key)

Để xóa một phần tử có khóa x ra khỏi từ điển T, trước hết ta cần xác định mã băm của x để xác định được rổ chứa x. Sau đó ta tiến hành tìm kiếm x trong rổ đó, nếu x có trong rổ, việc xóa x được thực hiện tương tự như việc xóa một phần tử ra khỏi danh sách liên kết đơn.

```
struct void delete(char *x, NodeT *T[N]){
    struct member *p, *q;
    int h = hashCode(x);
    p=T[h]; // bat dau tim kiem
    if (p=NULL){
        printf("Ro RONG, x khong co trong T");

    } else{
        if(p->key==x){// xoa dau ds
            T[h]=p->next->next;
            free(p);
        } else{
            // tim thay x thi stop
            while(p->key != x){
                q=p; // q tro den truoc ptu can xoa
                p=p->next;
            }
            if(p=NULL){ // khong thay x
                printf("x khong co trong T");
            }else{ // xoa x
                p=p->next;
                free(p)
            }
        }
    }
}
```

• Bảng băm đóng

Bảng băm đóng lưu giữ các phần tử của từ điển ngay trong mảng (các phần tử của “rổ” i lưu trong chính phần tử thứ i của mảng) chứ không dùng mảng để lưu trữ các con trỏ trỏ tới đầu của các danh sách liên kết – “rổ”.

Tương tự như băm mở, trong bảng băm đóng “rổ” thứ i chứa phần tử có giá trị băm là i, nhưng vì có thể có nhiều phần tử có cùng giá trị băm nên ta sẽ gặp trường hợp sau: ta muốn đưa vào “rổ” i một phần tử x nhưng “rổ” này đã bị chiếm bởi một

phần tử y nào đó => gây ra đụng độ. Như vậy khi thiết kế một bảng băm đóng ta phải có cách để giải quyết sự đụng độ này. Cách giải quyết đụng độ đó gọi là băm lại (rehash).

Chi tiết về phương pháp băm đóng bạn đọc có thể tham khảo các tài liệu tham khảo đã chỉ ra trong chương này

9.5 Bài tập luyện tập

Bài 1: Cho một tập hợp chứa các phần tử có khóa là các số nguyên. Yêu cầu:

1. Cài đặt các cấu trúc dữ liệu tập hợp bởi véc tơ bit.
2. Thực hiện các thao tác cơ bản trên tập hợp:
 - Nhập dữ liệu cho tập hợp
 - Hiển thị dữ liệu
 - Tìm kiếm một phần tử trong tập hợp
 - Hợp hai tập hợp A, B thành tập hợp C
 - Tìm giao của 2 tập hợp A, B
 - Tìm hiệu của 2 tập hợp A, B

Bài 2 : Cho một từ điển chứa 100 từ trong ngôn ngữ tiếng Anh. Yêu cầu:

1. Biểu diễn từ điển bởi CTDL bảng băm mở.
2. Thực hiện các thao tác sau trên từ điển:
 1. Thiết kế hàm băm
 2. Thêm một từ vào từ điển
 3. Xóa một từ ra khỏi từ điển
 4. Sửa một từ trong từ điển
 5. Tìm kiếm một từ trong từ điển
 6. Nhập và hiển thị dữ liệu trong từ điển.